Combinational Logic Circuit Design

Digital circuits can be generally classified into Combinational and Sequential, the difference being the former does not depend on time, aka instant output, while the other depends on time, by a clock signal (regular pulses of signal by the computer)

Designing Logic Circuits

- Generally, the steps of designing logic circuits are
  - Defining Requirements - defining the relationships between the inputs and output
  - Defining Truth Table - mapping out all possible inputs, and their corresponding outputs
  - Obtaining & Simplifying - finding a (simple) boolean expression that follows the truth table
  - Implementing/Building Circuit - implement based on the boolean expression
  - Verifying Truth Table - check results
- The main part of this process is defining the truth table, and obtaining and simplifying the boolean expression.
- Truth Table Writing
  - Lets say we want to make a circuit that can add 3 bits together.
  - We know that we should have 3 inputs for each of the 3 bits, and we should have 2 inputs, as the maximum output is 1+1+1 = 11 (2 bit output)
  - Thus, we can consider all inputs and their corresponding outputs.

A	B	C	Y	X
0	0	0	0	0
1	0	0	0	1
0	1	0	0	1
0	0	1	0	1
1	1	0	1	0
0	1	1	1	0
1	0	1	1	0
1	1	1	1	1

- Obtaining & Simplifying
  - Lets say we have the following truth table, with 2 outputs.

A	B	C	X	Y
0	0	0	0	1
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

  - Consider output X first. Notice how it outputs 1 when ABC is 101 or 110.

A	B	C	X
1	0	1	1
1	1	0	1

  - We can first consider how we can get an output when A B and C are 1 0 and 1 respectively.
  - We can use an AND gate, and invert any 0s like so: $A\bar{B}C$

Notice how when we input 101, we get $1 \cdot \bar{0} \cdot 1 = 1 \cdot 1 \cdot 1 = 1$

We can do the same for 110: $AB\bar{C}$.

Thus, since we want X to output 1 when either$A\bar{B}C$ or $AB\bar{C}$ is 1, we use an or gate:

$$X = A\bar{B}C + AB\bar{C}$$

We have now obtained a boolean expression for X, and we can now simplfy it to minimise the logic gates used:

$$\begin{aligned} X &= A\bar{B}C + AB\bar{C} \\ &= A(\bar{B}C + B\bar{C}) \\ &= A(B \oplus C) \end{aligned}$$

▼ Example with Output Y

We notice that majority of the outputs are 1, only 3 give 0, so we will focus on those

A	B	C	Y
0	1	0	0
1	0	1	0

Firstly, notice how we are focusing on 0s now, so ABC=010 should give 0, not 1.

thus, we need an operation that gives 0 for ABC = 010, which can be done with an OR gate: $A + \bar{B} + C$.

Recall that only when all inputs of the OR gate are 0, the output is 0. We can also show that OR gates work with De Morgan's Theorems and our previous method of using AND gates:

$$\overline{\bar{A}B\bar{C}} = \bar{\bar{A}} + \bar{B} + \bar{\bar{C}} = A + \bar{B} + C$$

Again, we need another operation that gives 0 when either input is 0, which is an AND gate.

$$Y = (A + \bar{B} + C)(\bar{A} + B + \bar{C})$$

Notice how when we focus on different outputs, we "swap" between AND and OR gates.

We can then simplify:

$$\begin{aligned} Y &= (A + \bar{B} + C)(\bar{A} + B + \bar{C}) \\ &= A\bar{A} + AB + A\bar{C} + \bar{B}\bar{A} + \bar{B}B + \bar{B}\bar{C} + C\bar{A} + CB + C\bar{C} \\ &= AB + A\bar{C} + \bar{B}\bar{A} + \bar{B}\bar{C} + C\bar{A} + CB \end{aligned}$$

From here, any common factors we remove wont significantly simplify the expression, so we can use the consensus theorem:

$$X\bar{Y} + YZ + XZ = X\bar{Y} + YZ$$

The main idea is that 3 terms that each have 2 inputs, and one of the inputs is negated in one but not in the other, the one without the negated input is removed (in this case, XZ as it has no Y term)

▼ Proof

$$\begin{aligned} X\bar{Y} + YZ + XZ &= X\bar{Y} + YZ + XZ(1) \\ &= X\bar{Y} + YZ + XZ(Y + \bar{Y}) \\ &= X\bar{Y} + YZ + XZY + XZ\bar{Y} \\ &= X\bar{Y} + XZ\bar{Y} + YZ + XZY \\ &= X\bar{Y}(1 + Z) + YZ(1 + X) \\ &= X\bar{Y} + YZ \end{aligned}$$

Using this theorem:

$$
\begin{aligned}
&= AB + A\bar{C} + \bar{B}\bar{A} + \bar{B}\bar{C} + C\bar{A} + CB \\
&= \left[AB + \bar{B}\bar{C} + \cancel{A\bar{C}}\right] + \bar{B}\bar{A} + C\bar{A} + CB \\
&= AB + \bar{B}\bar{C} + \bar{B}\bar{A} + C\bar{A} + CB \\
&= \left[AB + C\bar{A} + \cancel{CB}\right] + \bar{B}\bar{C} + \bar{B}\bar{A} \\
&= AB + C\bar{A} + \bar{B}\bar{C} + \bar{B}\bar{A} \\
&= AB + \left[C\bar{A} + \bar{B}\bar{C} + \cancel{\bar{B}\bar{A}}\right] \\
&= AB + \bar{A}C + \bar{B}\bar{C}
\end{aligned}
$$

apparently this isnt in the syllabus????????? or at least isnt mentioned??????????????????? wtf !!!!!!!!!!!!1

Karnaugh Maps (K-Maps)

Another way to form simplified boolean expressions from truth tables is with Karnaugh Maps.

Karnaugh Maps provide a 2D mapping of all the possible outputs, which allows a visual way to obtain simplified boolean expressions.

Constructing K-Maps

Each axis of the map represents either 1 or two inputs, and each cell on the map represents the combined output of them. For example:

B\A	A′ (0)	A (1)
B′ (0)	0	1
B (1)	1	0

Notice how the K-map presents the same information as a truth table. The first column and row represents the input space of each input, where A′ refers to 0, and A refers to 1.

C\AB	A′B′ (00)	A′B (01)	AB (11)	AB′ (10)
C′ (0)	1	1	1	0
C (1)	0	1	0	1

For 3 input K-maps, one of the axes represents 2 inputs, giving 4 possible outputs for AB (AND gate). However, across the axes, only 1 bit can change at a time
The K-Map still presents all 8 possible outputs for 3 inputs , e.g. when ABC=010 gives 1

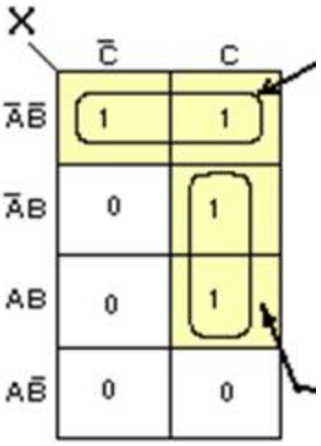
CD\AB	A′B′	A′B	AB	AB′
C′D′	1	0	0	1
C′D	0	1	1	0
CD	0	1	1	0
CD′	0	0	1	0

Same with 4 inputs, each axis represents 2 inputs, giving 16 outputs in total, the same as the truth table.

With more than 4 inputs, it is impractical to use K-maps.

After mapping our truth table to a Kmap, we can group regions of 1 together to form a simplified boolean expression

Consider the following k-map:



The way we obtain the boolean expression from the map is by grouping 1s together, by circling/looping them.

We have to make sure we only circle 2^n number of 1s (aka only groups of 1, 2, 4 8, etc.)

In this case, we can circle 2 pairs of 1s.

The way we interpret these groups is by what input has itself and its inverse.

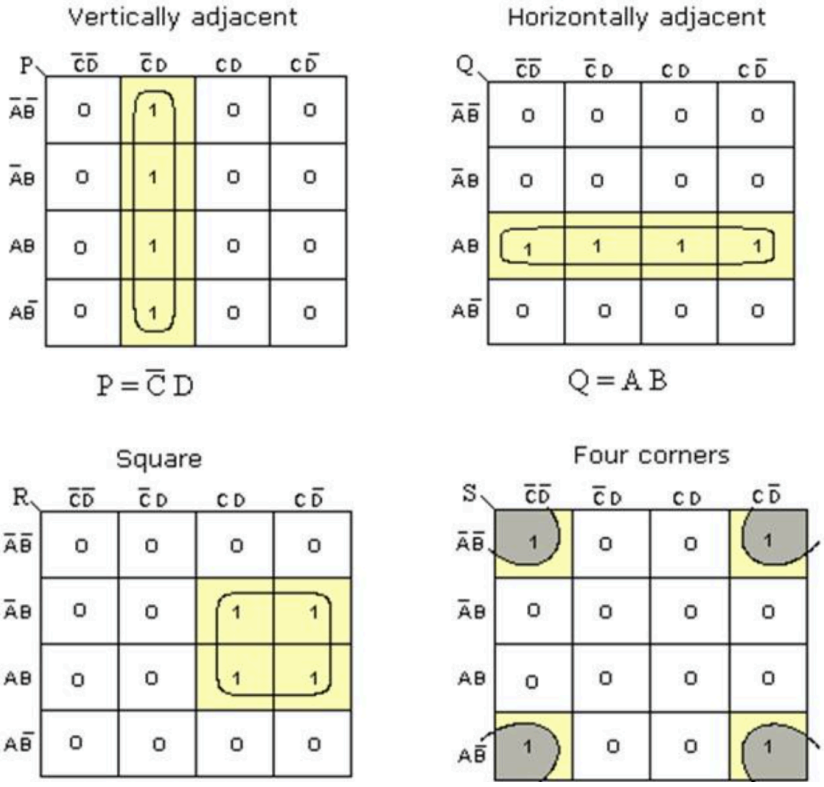
For example, the outputs $\bar{A}\bar{B}\bar{C}$ and $\bar{A}\bar{B}C$ are grouped, thus the group represents $\bar{A}\bar{B}$ (we remove the different variable) This is just a visual way of adding the two outputs together (normal and inverse cancel)

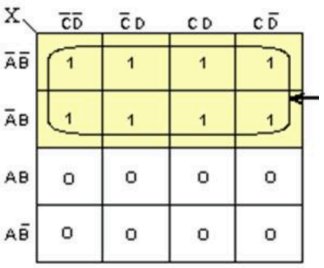
After finding the next group (BC), we just add the groups together to form the boolean expression for the output:

$$X = \bar{A}\bar{B} + BC = \overline{A+B} + BC$$

We can also simplify it to use NOR gates instead of 2 NOT gates, for easier implementation.

When looping 4 or 8 ones, we can either loop them in a line, or in a square





$X = \bar{A}$

Notice how when we loop 2^n 1s, we eliminate n inputs from each loop.

2 important things to note on looping is that:

Loops can overlap, usually to form as many pairs as possible, to minimize isolated 1s.

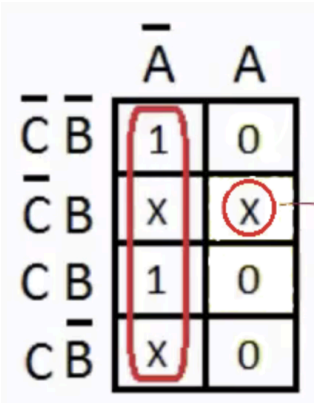
The K-map itself can loop on itself, allowing loops to cross from the right edge to the left (like the 4 corners example)

Don't Cares

Some systems may not need the full output space, thus we can do not need to care about these states

For example, with BCD, each 4 bit digit goes up to 1001, states like 1010 and up would not exist and thus we do not need to care about their outputs.

We can represent these don't care states as Xs on our K-map, acting as "wild cards", which we can let it be 0 or 1 whenever convenient



For example, for this K-map, it is easier to let the left 2 Xs be 1 and the right 1 be 0, because a quad can be looped together easily to maximise the variables eliminated

SOMETHING:

Octet to 7 segment display truth table

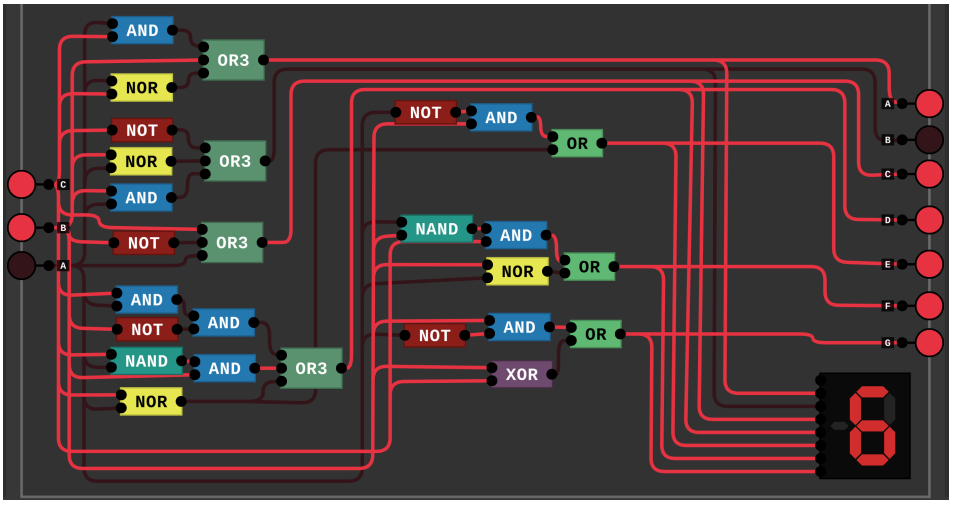
N	C	B	A		a	b	c
0	0	0	0		1	1	1
1	0	0	1		0	1	1
2	0	1	0		1	1	0
3	0	1	1		1	1	1
4	1	0	0		0	1	1

5	1	0	1		1	0	1
6	1	1	0		1	0	1
7	1	1	1		1	1	1

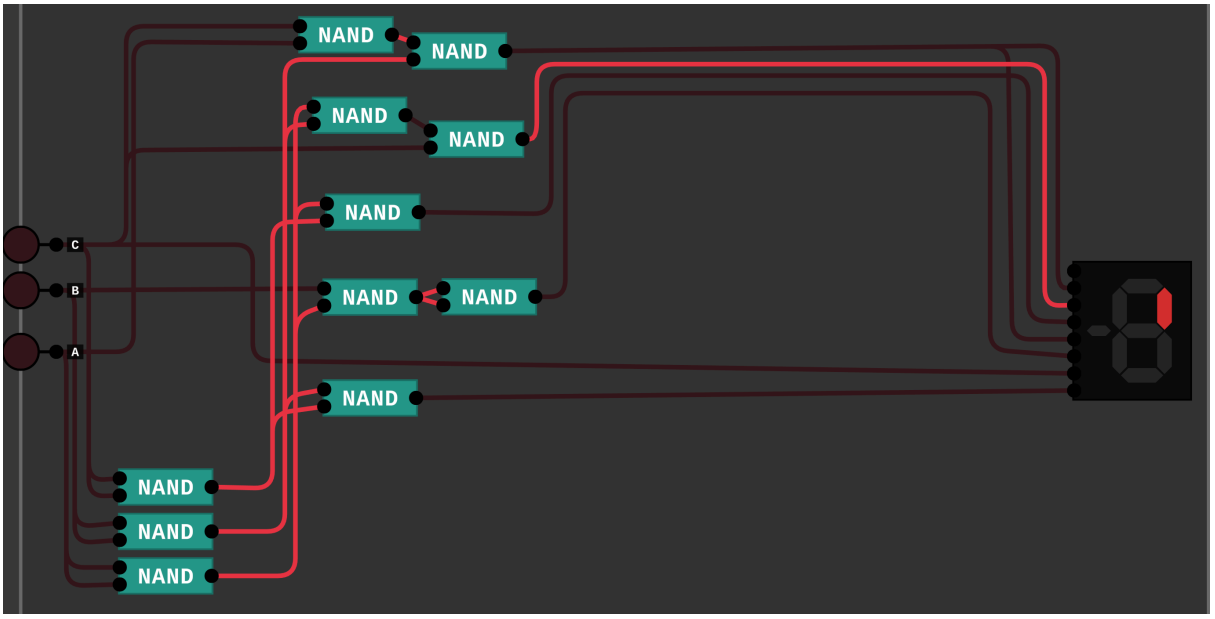
N	C	B	A		a	b	c
0	0	0	0		1	1	1
1	0	0	1		x	x	x
2	0	1	0		x	x	x
3	0	1	1		1	1	1
4	1	0	0		0	1	1
5	1	0	1		1	0	1
6	1	1	0		1	0	1
7	1	1	1		1	1	1

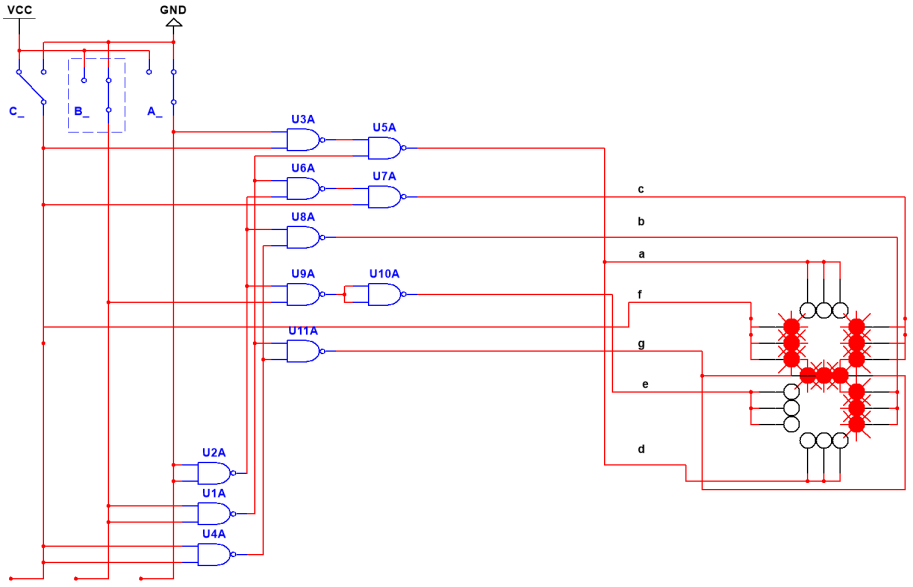
$$a = B + \overline{A + C} + AC$$
$$b = \bar{C} + \overline{B + A} + BA$$
$$c = A + \bar{B} + C$$
$$d = A\bar{B}C + B\overline{AC} + \overline{A + C}$$
$$e = B\bar{A} + \overline{A + C}$$
$$f = \overline{B + A} + C\overline{AB}$$
$$g = B \oplus C + B\bar{A}$$

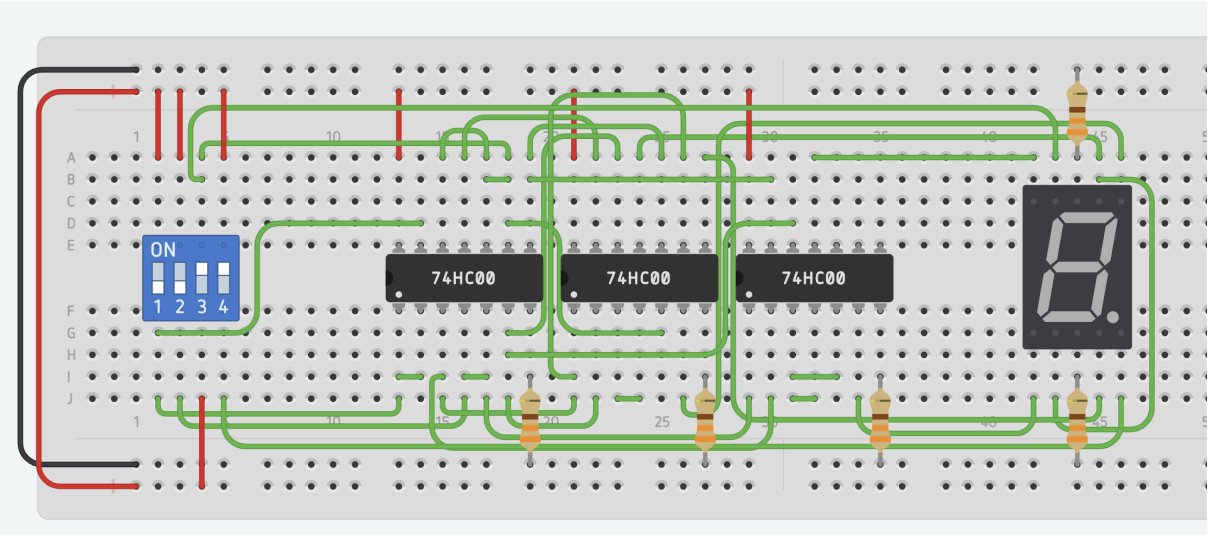
Implementation:



NAND implementation of octet to 7 segment (dont care 0 and 7)


NAND
NAND
NAND
NAND
NAND
NAND
NAND
NAND
NAND
NAND
NAND
C
B
A


VCC
GND
C_
B_
A_
U3A
U5A
U6A
U7A
U8A
U9A
U10A
U11A
U2A
U1A
U4A
c
b
a
f
g
e
d


ON
1 2 3 4
74HC00
74HC00
74HC00

$$
\begin{aligned}
b &= \bar{B}\bar{A} + BA \\
&= \overline{\overline{\bar{B}\bar{A} + BA}} \\
&= \overline{\overline{\bar{B}\bar{A}} \cdot \overline{BA}}
\end{aligned}
$$

$$
\begin{aligned}
a &= A + \bar{C} + B \\
&= \overline{\overline{A + \bar{C} + B}} \\
&= \overline{\bar{A} \cdot \bar{\bar{C}} \cdot \bar{B}} \\
&= \overline{\bar{A} \cdot C \cdot \bar{B}}
\end{aligned}
$$